

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐHQG THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BỘ MÔN: NHẬP MÔN DEVOPS

Đồ án 2: Xây dựng hệ thống CD

Giảng viên:

Nguyễn Thanh Quân
Lê Quốc Hòa
Trương Phước Hưng
Lê Giang Thanh

Mạch Quốc Tấn	23127115
Nguyễn Phúc Thọ	23127123
Nguyễn Lê Hồ Anh Khoa	23127211

Thành phố Hồ Chí Minh, ngày 14 tháng 5 năm 2026

Contents

1	Tổng quan Đồ án	4
1.1	Giới thiệu	4
1.1.1	Yêu cầu bắt buộc (6 điểm)	4
1.1.2	Yêu cầu nâng cao (2+2 điểm)	4
1.2	Kiến trúc Hệ thống	5
1.2.1	Kiến trúc tổng thể	5
1.2.2	Kiến trúc CI/CD Pipeline	6
1.2.3	Kiến trúc Service Mesh (Istio)	7
1.3	Sơ đồ	7
1.3.1	Sơ đồ tổng quan Namespace và Luồng traffic	7
1.3.2	Sơ đồ K3s Multi-node Cluster	7
1.3.3	Sơ đồ Service Mesh Topology (Kiali)	7
1.3.4	Sơ đồ ArgoCD GitOps	8
2	Cấu hình	9
2.1	Chuẩn bị môi trường	9
2.1.1	Lý do chọn K3s thay Minikube	9
2.1.2	Cài đặt K3s	9
2.1.3	Cài Ingress NGINX và cấu hình inotify	10
2.1.4	Thêm Worker Node vào Cluster	10
2.2	Flow Setup — CI/CD Pipeline	11
2.2.1	Cấu hình GitHub Secrets	11
2.2.2	Self-hosted Runner	11
2.2.3	Workflow CI (Yêu cầu 3)	11
2.2.4	Workflow Developer Build (Yêu cầu 4)	13
2.2.5	Workflow Cleanup (Yêu cầu 5)	13
2.2.6	Workflow Dev Deploy (Yêu cầu 6a)	14
2.2.7	Workflow Staging Deploy (Yêu cầu 6b)	14
2.3	Cấu hình ArgoCD (Nâng cao)	14
2.3.1	Cài đặt ArgoCD	14
2.3.2	Cấu hình ApplicationSet	14
2.4	Cấu hình Istio Service Mesh (Nâng cao)	15
2.4.1	Cài đặt Istio	15
2.4.2	Giải thích code cấu hình Istio	16
2.4.3	Ingress NGINX kết hợp Istio mTLS STRICT	17
2.5	Cấu hình Infrastructure	18

2.5.1	Setup cluster script	18
2.5.2	Cấu hình CoreDNS cho hostname nội bộ	18
3	Kiểm thử	19
3.1	Kiểm thử CI Pipeline (Yêu cầu 3)	19
3.1.1	Kịch bản test	19
3.1.2	Kết quả mong đợi	19
3.2	Kiểm thử Developer Build (Yêu cầu 4)	20
3.2.1	Kịch bản test	20
3.2.2	Kết quả	20
3.3	Kiểm thử Cleanup (Yêu cầu 5)	20
3.4	Kiểm thử Dev/Staging Deploy (Yêu cầu 6)	21
3.4.1	Test Dev Environment	21
3.4.2	Test Staging Environment	21
3.5	Kiểm thử Istio mTLS (Nâng cao)	21
3.5.1	Kiểm tra mTLS đang hoạt động	21
3.5.2	Kiểm tra Authorization Policy	22
3.5.3	Kiểm tra Retry Policy	22
3.5.4	Kiali Topology	22
3.6	Kiểm thử End-to-End	23
4	Vấn đề gặp phải và Cách khắc phục	24
4.1	Vấn đề Infrastructure	24
4.1.1	PostgreSQL cluster không tạo được — Helm template syntax error	24
4.1.2	Strimzi Kafka — Chuỗi 3 vấn đề tương thích	24
4.1.3	Promtail CrashLoopBackOff — “too many open files”	25
4.1.4	ArgoCD ApplicationSet Controller CrashLoopBackOff	26
4.1.5	Loki chart mới yêu cầu schema_config	26
4.1.6	Prometheus/Grafana — leaked secrets validation	26
4.1.7	OpenTelemetry Collector — receiver/exporter “loki” không tồn tại	27
4.2	Vấn đề K3s và Chuyển đổi từ Minikube	27
4.2.1	Cluster quá tải — API server ngừng hoạt động (Minikube)	27
4.2.2	K3s agent không start — port 6444 đang bị chiếm	27
4.2.3	Firewall Fedora block inter-pod traffic	28
4.3	Vấn đề Istio Service Mesh	28
4.3.1	502 Bad Gateway sau khi bật mTLS STRICT	28
4.3.2	Kiali cảnh báo KIA0601 — Port name sai format	28
4.3.3	Kiali lỗi DestinationRule wildcard host	28
4.3.4	AuthorizationPolicy không match workload	29
4.4	Tổng kết	29

Chapter 1

Tổng quan Đề án

1.1 Giới thiệu

Đề án 2 yêu cầu xây dựng một hệ thống CI/CD (Continuous Integration / Continuous Delivery) hoàn chỉnh để triển khai, vận hành và giám sát ứng dụng microservice **YAS** — **Yet Another Shop** trên nền tảng Kubernetes.

YAS là một ứng dụng thương mại điện tử mã nguồn mở được xây dựng theo kiến trúc microservice, gồm hơn 20 service viết bằng Java/Spring Boot và Next.js. Ứng dụng sử dụng các công nghệ hiện đại như Kafka, Elasticsearch, Keycloak, OpenTelemetry, Grafana, Loki, Prometheus và Tempo.

1.1.1 Yêu cầu bắt buộc (6 điểm)

- YC1. Sử dụng image có tag `main/latest` cho tất cả service làm mặc định.
- YC2. Xây dựng K8s cluster ít nhất 1 Master + 1 Worker (hoặc Minikube/K3s).
- YC3. **Pipeline CI**: Khi developer commit lên bất kỳ branch nào, tự động build image với tag = `commit_id` và push lên Docker Hub.
- YC4. **Job CD developer_build**: Developer chỉ định service + branch muốn test, pipeline deploy service đó với image mới, các service còn lại dùng `latest`. Cung cấp domain để developer truy cập kiểm tra.
- YC5. **Job CD xóa deployment** từ yêu cầu 4.
- YC6. **Pipeline CI/CD** tự động: (a) push `main` → deploy vào namespace `dev`; (b) tạo tag `vX.Y.Z` → build image mới, deploy vào namespace `staging`.

1.1.2 Yêu cầu nâng cao (2+2 điểm)

- **ArgoCD (2đ)**: Sử dụng ArgoCD thay thế deploy trực tiếp để quản lý GitOps cho môi trường `dev` và `staging`.
- **Service Mesh / Istio (2đ)**: Cấu hình mTLS giữa các service, Authorization Policy, Retry Policy, và visualize topology qua Kiali.

1.2 Kiến trúc Hệ thống

1.2.1 Kiến trúc tổng thể

Hệ thống được triển khai trên **K3s v1.32.5** chạy trên máy Fedora Linux 43 với cấu hình 32 CPU / 32GB RAM / 952GB NVMe. Cluster bao gồm 1 control plane node và 2 worker node.

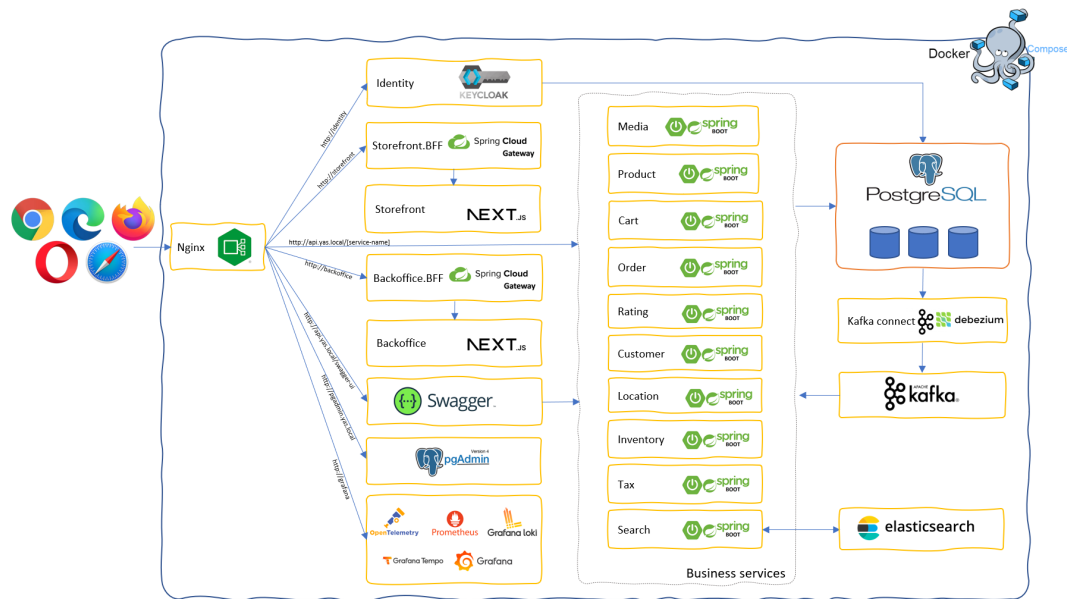


Figure 1.1: Kiến trúc tổng thể YAS microservice

Toàn bộ hệ thống được tổ chức thành các lớp như sau:

Table 1.1: Phân tầng hệ thống

Tầng	Namespace	Thành phần
Ingress	ingress-nginx	ingress-nginx-controller (LoadBalancer, klipper-lb)
Service Mesh	istio-system	istiod, istio-ingressgateway, istio-egressgateway, Kiali
Applications	yas	20+ microservices (cart, order, product, customer, ...)
GitOps	argocd	ArgoCD server, repo-server, applicationset-controller
Identity	keycloak	Keycloak 26.0.2 (OAuth2/OIDC)
Database	postgres	PostgreSQL (Zalando operator), pgAdmin
Messaging	kafka	Kafka 4.1.0 KRaft mode (Strimzi 0.51), AKHQ
Search	elasticsearch	Elasticsearch standalone
Cache	redis	Redis (session cache cho BFF)
Observability	observability	Prometheus, Grafana, Loki, Tempo, OpenTelemetry

1.2.2 Kiến trúc CI/CD Pipeline

Pipeline CI/CD được xây dựng hoàn toàn trên **GitHub Actions**:

- **CI** (`npt-ci.yml`): Chạy trên GitHub-hosted runner, không cần truy cập cluster.
- **CD** (các workflow deploy): Chạy trên **self-hosted runner** đặt tại máy chủ để có thể kết nối trực tiếp vào K3s cluster qua `127.0.0.1:6443`.

Luồng CI/CD Pipeline

1. Developer push code lên branch bất kỳ
2. **CI workflow** trigger: detect service thay đổi → build Maven/Docker → push image (`image:commit_id`) lên Docker Hub
3. Nếu push lên `main`: tag thêm `:latest` → trigger **Dev Deploy workflow** → ArgoCD sync → deploy vào namespace `dev`
4. Nếu tạo tag `v*`: trigger **Staging Deploy workflow** → build image với tag version → deploy vào namespace `staging`
5. Developer muốn test branch riêng: chạy **developer_build** workflow thủ công → chỉ định service + branch → deploy vào namespace `developer`

1.2.3 Kiến trúc Service Mesh (Istio)

Toàn bộ traffic trong namespace **yas** đi qua **Istio sidecar proxy (Envoy)**, đảm bảo mTLS STRICT giữa mọi service. Đặc biệt, **ingress-nginx** cũng được inject sidecar và tham gia vào mesh, cho phép duy trì mTLS ngay từ điểm vào hệ thống.

1.3 Sơ đồ

1.3.1 Sơ đồ tổng quan Namespace và Luồng traffic

Luồng traffic qua K3s cluster

Browser / Curl

- ↓ DNS (/etc/hosts → 172.16.0.240)
- ↓ **klipper-lb** (K3s built-in LoadBalancer, port 80/443)
- ↓ **ingress-nginx-controller** (namespace: **ingress-nginx**, có Istio sidecar)
- ↓ Istio mTLS (ISTIO_MUTUAL via DestinationRule)
- ↓ **Microservice** (namespace: **yas**, pod có Envoy sidecar 2/2)
- ↓ Service-to-service: mTLS STRICT, Authorization Policy kiểm soát từng cặp

1.3.2 Sơ đồ K3s Multi-node Cluster

Table 1.2: Cấu hình K3s Cluster

Node	Role	Mô tả
fedora	control-plane, master	Máy chủ chính; chạy etcd, API server, scheduler, controller-manager, istiod, ArgoCD
worker-01	worker	Máy worker 1; chạy workload pods
worker-02	worker	Máy worker 2; chạy workload pods

Worker node được thêm vào cluster bằng K3s agent:

```
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.32.5+k3s1 \
  K3S_URL="https://${CONTROL_PLANE_IP}:6443" \
  K3S_TOKEN="${TOKEN}" \
  sh -
```

Listing 1.1: Cài K3s agent trên worker node

1.3.3 Sơ đồ Service Mesh Topology (Kiali)

Kiali Dashboard hiển thị topology của service mesh trong namespace **yas**. Mỗi mũi tên thể hiện một kết nối có mTLS được xác thực. Màu xanh = healthy, màu vàng = có cảnh báo cấu hình (KIA0601 — đã được fix).

Cách xem Kiali Topology

```
# Port-forward Kiali dashboard
kubectl port-forward svc/kiali -n istio-system 20001:20001

# Truy cap: http://localhost:20001
# Menu: Graph -> namespace "yas" -> Display: Security (bat mTLS
      badges)
```

1.3.4 Sơ đồ ArgoCD GitOps

ArgoCD sử dụng mô hình **ApplicationSet** để tự động tạo Application cho từng environment (dev, staging) từ một template duy nhất:

```
# argocd/applications/dev-appset.yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: yas-dev-appset
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - service: cart
          - service: order
          # ... 20+ services
  template:
    spec:
      project: yas
      source:
        repoURL: https://github.com/NPT-102/yas-2
        targetRevision: main
        path: k8s/charts/\\{\\{ service \\}\\}
      destination:
        namespace: dev
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
```

Listing 1.2: Cấu trúc ArgoCD ApplicationSet

Chapter 2

Cấu hình

2.1 Chuẩn bị môi trường

2.1.1 Lý do chọn K3s thay Minikube

Ban đầu đề án được thực hiện trên Minikube. Sau quá trình triển khai, nhóm quyết định chuyển sang **K3s v1.32.5** vì các lợi thế vượt trội:

Table 2.1: So sánh Minikube và K3s

Tiêu chí	Minikube	K3s
Cơ chế chạy	Docker container (Docker-in-Docker)	Systemd service (native process)
RAM overhead	~2-3 GB	~512 MB
Port 80/443	Cần <code>minikube tunnel</code>	Tự động (klipper-lb)
Auto-start sau reboot	Không	Có (<code>systemctl enable k3s</code>)
inotify (Promtail, Kafka)	Phải set lại sau restart	Persist trong <code>/etc/sysctl.d/</code>
Worker node	Docker container giả lập	Máy thật (VM/máy khác)

2.1.2 Cài đặt K3s

```
# Cài K3s v1.32.5 stable LTS, tắt Traefik (dùng ingress-nginx thay the
)
curl -sfL https://get.k3s.io | INSTALL_K3S_VERSION=v1.32.5+k3s1 sh -s
- \
--disable=traefik \
--write-kubeconfig-mode=644

# Cấu hình kubeconfig
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
```

```
# Kiểm tra
kubectl get nodes
# NAME          STATUS    ROLES                  AGE     VERSION
# fedora        Ready     control-plane,master   1m      v1.32.5+k3s1
```

Listing 2.1: Cài K3s v1.32.5 (disable Traefik)

Lưu ý quan trọng về version

Không dùng K3s v1.35.x — có bug `cloud-controller-manager` exit ngay khi gặp 403 Forbidden lúc bootstrap RBAC (race condition), khiến K3s crash loop vĩnh viễn. Dùng v1.32.5 (LTS stable).

2.1.3 Cài Ingress NGINX và cấu hình inotify

```
# Cài ingress-nginx
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/\
controller-v1.11.0/deploy/static/provider/cloud/deploy.yaml

# Tăng giới hạn inotify (can cho Kafka, Promtail)
sudo sysctl -w fs.inotify.max_user_watches=524288
sudo sysctl -w fs.inotify.max_user_instances=512

# Persist qua reboot (chỉ cần set 1 lần với K3s)
echo "fs.inotify.max_user_watches=524288" | sudo tee -a /etc/sysctl.d/99-k3s.conf
echo "fs.inotify.max_user_instances=512" | sudo tee -a /etc/sysctl.d/99-k3s.conf
sudo sysctl --system
```

Listing 2.2: Cài ingress-nginx và tăng giới hạn inotify

2.1.4 Thêm Worker Node vào Cluster

```
# Trên control plane: lấy token
sudo cat /var/lib/rancher/k3s/server/node-token

# Mở firewall trên control plane
sudo firewall-cmd --permanent --add-port=6443/tcp
sudo firewall-cmd --permanent --add-port=8472/udp
sudo firewall-cmd --reload

# Trên worker: cài K3s agent
curl -sL https://get.k3s.io | INSTALL_K3S_VERSION=v1.32.5+k3s1 \
  K3S_URL="https://${CONTROL_PLANE_IP}:6443" \
  K3S_TOKEN="${TOKEN}" sh -

# Kiểm tra trên control plane
kubectl get nodes
# NAME          STATUS    ROLES    AGE
```

```
# fedora      Ready    control-plane,master    2d
# worker-01   Ready    <none>                  5m
# worker-02   Ready    <none>                  3m
```

Listing 2.3: Join worker node vào K3s cluster

2.2 Flow Setup — CI/CD Pipeline

2.2.1 Cấu hình GitHub Secrets

Ba secrets bắt buộc cần cấu hình trong GitHub repository:

Table 2.2: GitHub Secrets

Secret	Mô tả
DOCKER_USER	Username Docker Hub — dùng để đặt tên image và login
DOCKER_PASS	Access Token Docker Hub — dùng để push image
KUBE_CONFIG	Kubeconfig base64 — dùng để kubectl deploy lên K3s

```
# Encode kubeconfig thành base64
cat ~/.kube/config | base64 -w 0
# Copy output -> GitHub Settings -> Secrets -> KUBE_CONFIG
```

Listing 2.4: Lấy KUBE_CONFIG cho K3s

2.2.2 Self-hosted Runner

Vì K3s API server chạy tại 127.0.0.1:6443, GitHub-hosted runner trên cloud không thể kết nối được. Nhóm cài **self-hosted runner** trực tiếp trên máy chủ:

```
mkdir -p ~/actions-runner && cd ~/actions-runner
curl -o actions-runner-linux-x64-2.322.0.tar.gz -L \
  https://github.com/actions/runner/releases/download/v2.322.0/\
  actions-runner-linux-x64-2.322.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.322.0.tar.gz
./config.sh --url https://github.com/NPT-102/yas-2 --token <TOKEN>

# Chạy background
nohup ./run.sh > runner.log 2>&1 &
```

Listing 2.5: Cài và chạy self-hosted runner

2.2.3 Workflow CI (Yêu cầu 3)

File: .github/workflows/npt-ci.yml

Luồng hoạt động:

1. Trigger: push lên bất kỳ branch nào (branches: '**')

2. **Job detect-changes:** So sánh git diff để phát hiện service nào thay đổi
3. **Job build-java:** Chạy mvn clean install, build Docker image, push lên Docker Hub với tag = 7 ký tự đầu commit SHA
4. **Job build-frontend:** Build Next.js qua multi-stage Dockerfile, push image
5. Nếu push vào main: tag thêm :latest

```

name: CI Build and Push
on:
  push:
    branches: ['**']

jobs:
  detect-changes:
    runs-on: ubuntu-latest
    outputs:
      java_services: ${ steps.filter.outputs.java_services }
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 2
      - name: Detect changed services
        id: filter
        run: |
          CHANGED=$(git diff --name-only HEAD~1 HEAD)
          for svc in $JAVA_SERVICES; do
            if echo "$CHANGED" | grep -q "^${svc}/"; then
              JAVA+=("${svc}\n")
            fi
          done
          echo "java_services=[$(IFS=,; echo "${JAVA[*]}")]" >>
            $GITHUB_OUTPUT

  build-java:
    needs: detect-changes
    if: needs.detect-changes.outputs.java_services != '[]'
    strategy:
      matrix:
        service: ${ fromJson(needs.detect-changes.outputs.
          java_services) }
    steps:
      - name: Build and Push Docker Image
        run: |
          COMMIT_ID=${GITHUB_SHA:0:7}
          IMAGE=${ secrets.DOCKER_USER }/${ matrix.service }
          TAGS="-t ${IMAGE}:${COMMIT_ID}"
          if [ "${ github.ref_name }" = "main" ]; then
            TAGS="$TAGS -t ${IMAGE}:latest"
          fi
          docker buildx build --push $TAGS ./${ matrix.service }

```

Listing 2.6: CI workflow — detect changes và build (trích)

2.2.4 Workflow Developer Build (Yêu cầu 4)

File: `.github/workflows/npt-developer_build.yml`

Developer kích hoạt thủ công qua `workflow_dispatch`, nhập:

- `service_name`: service muốn test (dropdown list)
- `branch_name`: tên branch đang phát triển

Luồng hoạt động:

1. Lấy commit SHA cuối của branch được chọn
2. Checkout code của branch đó → build + push image với tag = SHA
3. Deploy tất cả service vào namespace `developer`:
 - Service được chọn: dùng image mới build, expose qua NodePort
 - Các service còn lại: dùng image `:latest`
4. In ra GitHub Step Summary: NodePort, Worker IP, hướng dẫn `/etc/hosts`

```
- name: Deploy all services
  run: |
    for svc in $SERVICES; do
      if [ "${svc}" = "${CUSTOM_SERVICE}" ]; then
        # Deploy custom branch image
        helm upgrade --install ${svc} ./${svc} \
          --namespace developer --create-namespace \
          --set backend.image.repository=${DOCKER_USER}/${svc} \
          --set backend.image.tag=${CUSTOM_TAG} \
          --set backend.service.type=NodePort \
          --set backend.ingress.host="api.developer.yas.local.com"
      else
        # Deploy latest image
        helm upgrade --install ${svc} ./${svc} \
          --namespace developer --create-namespace \
          --set backend.image.tag=latest
      fi
    done
```

Listing 2.7: Developer build — deploy service với custom image (trích)

2.2.5 Workflow Cleanup (Yêu cầu 5)

File: `.github/workflows/npt-cleanup.yml`

Developer chạy workflow, nhập `yes` để xác nhận → xóa toàn bộ namespace `developer`.

```
kubectl delete namespace developer --ignore-not-found=true
```

Listing 2.8: Cleanup deployment

2.2.6 Workflow Dev Deploy (Yêu cầu 6a)

File: `.github/workflows/npt-dev-deploy.yml`

Trigger: **tự động** khi push vào main. Deploy tất cả service vào namespace `dev`. ArgoCD ApplicationSet đồng bộ Helm chart từ Git → tự động sync khi `main` thay đổi.

2.2.7 Workflow Staging Deploy (Yêu cầu 6b)

File: `.github/workflows/npt-staging-deploy.yml`

Trigger: tạo tag `v*` trên branch `main`.

```
git checkout main && git pull
git tag v1.0.0
git push origin v1.0.0
# Pipeline tu dong build tat ca service voi tag v1.0.0
# Push images: <DOCKER_USER>/<service>:v1.0.0
# Deploy vao namespace staging
```

Listing 2.9: Tạo staging release

2.3 Cấu hình ArgoCD (Nâng cao)

2.3.1 Cài đặt ArgoCD

```
kubectl create namespace argocd
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/
  manifests/install.yaml \
  --server-side # Can thiet vi CRD ArgoCD lon hon 256KB

# Truy cap UI qua port-forward
kubectl port-forward svc/argocd-server -n argocd 9090:443 &
# https://localhost:9090 (admin / <mat-khau-lay-tu-secret>)
```

Listing 2.10: Cài ArgoCD với server-side apply

2.3.2 Cấu hình ApplicationSet

ArgoCD ApplicationSet tự động tạo Application cho từng service trong mỗi environment.

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: yas-dev-appset
  namespace: argocd
spec:
  generators:
    - list:
        elements:
          - service: cart
          - service: order
          - service: product
```

```
# ... them 17 services
template:
  metadata:
    name: 'yas-dev-\\{\\{service\\}\\}'
  spec:
    project: yas
    source:
      repoURL: https://github.com/NPT-102/yas-2
      targetRevision: main
      path: 'k8s/charts/\\{\\{service\\}\\}'
    destination:
      server: https://kubernetes.default.svc
      namespace: dev
    syncPolicy:
      automated:
        prune: true
        selfHeal: true
      syncOptions:
        - CreateNamespace=true
```

Listing 2.11: ArgoCD ApplicationSet cho môi trường dev

2.4 Cấu hình Istio Service Mesh (Nâng cao)

2.4.1 Cài đặt Istio

```
# Cài Istio với resource limits tối ưu (istio-overlay.yaml)
./istio-1.24.3/bin/istioctl install -f istio/istio-overlay.yaml -y

# Bật sidecar injection
kubectl label namespace yas istio-injection=enabled
kubectl label namespace ingress-nginx istio-injection=enabled # QUAN
TRONG!

# Restart để nhận sidecar (2/2 containers)
kubectl rollout restart deployment -n yas
kubectl rollout restart deployment ingress-nginx-controller -n ingress
-nginx

# Cài addons
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release
-1.24/samples/addons/prometheus.yaml
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release
-1.24/samples/addons/grafana.yaml
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release
-1.24/samples/addons/kiali.yaml
```

Listing 2.12: Cài Istio với profile tối ưu

2.4.2 Giải thích code cấu hình Istio

PeerAuthentication — bắt buộc mTLS

```
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
  name: default-mtls
  namespace: yas
spec:
  mtls:
    mode: STRICT # Bat buoc mTLS cho toan bo traffic vao namespace
    yas
    # ingress-nginx co sidecar -> tu dung mTLS qua
    DestinationRule
```

Listing 2.13: istio/peer-authentication.yaml

Giải thích: STRICT mode yêu cầu mọi kết nối vào namespace `yas` phải dùng mTLS. Điều này có nghĩa là chỉ các client *trong mesh* (có Envoy sidecar) mới giao tiếp được. Đây là lý do ingress-nginx cũng phải được inject sidecar.

DestinationRule — cấu hình TLS mode

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: mtls-cart
  namespace: yas
spec:
  host: cart.yas.svc.cluster.local
  exportTo:
    - "." # Namespace yas
    - "ingress-nginx" # Quan trong: ingress-nginx phai thay duoc rule nay
  trafficPolicy:
    tls:
      mode: ISTIO_MUTUAL # Envoy tu quan ly chung chi, khong can
      config thu cong
```

Listing 2.14: istio/destination-rule.yaml (trích — 1 trong 21 rules)

Giải thích: `exportTo: ingress-nginx` cho phép sidecar trong namespace `ingress-nginx` nhìn thấy `DestinationRule` và tự dùng mTLS. Nếu thiếu field này, ingress-nginx proxy qua Pod IP (PassthroughCluster) thay vì service identity → mTLS STRICT fail.

AuthorizationPolicy — kiểm soát service-to-service

```
# Policy mac dinh: deny all
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: deny-all
  namespace: yas
```



```
spec: {}    # Không có rules = deny tất cả

---
# Chỉ cho phép order truy cập tax
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: allow-order-to-tax
  namespace: yas
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: tax    # Pod nào áp dụng policy này
  action: ALLOW
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/yas/sa/order"    # Chỉ order được phép
    - from:
        - source:
            principals:
              - "cluster.local/ns/ingress-nginx/sa/ingress-nginx"
```

Listing 2.15: istio/authorization-policy.yaml (trích)

VirtualService — Retry Policy

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: tax-retry
  namespace: yas
spec:
  hosts:
    - tax.yas.svc.cluster.local
  http:
    - retries:
        attempts: 3            # Thu lại 3 lần nếu lỗi
        perTryTimeout: 10s
        retryOn: 5xx,reset,connect-failure,retriable-4xx
      route:
        - destination:
            host: tax.yas.svc.cluster.local
```

Listing 2.16: istio/virtual-service-retry.yaml (trích)

Giải thích: Nếu service `tax` trả lỗi 5xx, Istio tự động retry tối đa 3 lần mà không cần code application xử lý retry. Điều này tăng độ bền của hệ thống.

2.4.3 Ingress NGINX kết hợp Istio mTLS STRICT

Thách thức lớn nhất: `ingress-nginx` ban đầu nằm ngoài mesh → gửi plain HTTP vào namespace `yas` → sidecar từ chối (STRICT mode) → 502 Bad Gateway.

Giải pháp 3 bước:

1. Inject sidecar vào `ingress-nginx` (label namespace + restart pod)
2. Export `DestinationRule` sang namespace `ingress-nginx`
3. Thêm annotation vào Ingress để proxy theo service identity (không theo Pod IP):

```
# k8s/charts/backend/templates/ingress.yaml
annotations:
  nginx.ingress.kubernetes.io/service-upstream: "true"
  nginx.ingress.kubernetes.io/upstream-vhost: "{{ .Chart.Name }}.yas.svc.cluster.local"
```

Listing 2.17: Annotation Ingress để ingress-nginx dùng service identity

Khi bật `service-upstream`, Envoy sidecar của ingress-nginx route traffic qua cluster name `outbound|80||cart.yas.svc.cluster.local` thay vì `PassthroughCluster` → `DestinationRule` được áp dụng → mTLS hoạt động.

2.5 Cấu hình Infrastructure

2.5.1 Setup cluster script

Script `k8s/deploy/setup-cluster.sh` tự động cài toàn bộ infrastructure:

```
cd k8s/deploy
./setup-cluster.sh      # ~15 phut
./setup-keycloak.sh    # ~5 phut
./setup-redis.sh       # ~1 phut
./deploy-yas-configuration.sh # ConfigMaps + Secrets
./deploy-yas-minimal.sh      # Deploy microservices
```

Listing 2.18: Chạy setup cluster

2.5.2 Cấu hình CoreDNS cho hostname nội bộ

Pods trong cluster không đọc được `/etc/hosts` của host → cần thêm hostname vào CoreDNS:

```
kubectl edit configmap coredns -n kube-system
# Thêm vào block "hosts /etc/coredns/NodeHosts {"
# 172.16.0.240 identity.yas.local.com
# 172.16.0.240 api.yas.local.com

kubectl rollout restart deployment/coredns -n kube-system
```

Listing 2.19: Thêm hostname vào CoreDNS

Chapter 3

Kiểm thử

3.1 Kiểm thử CI Pipeline (Yêu cầu 3)

3.1.1 Kịch bản test

```
# Tao branch moi
git checkout -b dev_tax_service

# Chinh sua code trong service tax
echo "// test change" >> tax/src/main/java/com/yas/tax/TaxApplication.
    java

# Commit va push
git add .
git commit -m "test: trigger CI pipeline for tax service"
git push origin dev_tax_service
```

Listing 3.1: Test CI pipeline

3.1.2 Kết quả mong đợi

1. Workflow CI Build and Push tự động trigger
2. Job detect-changes phát hiện thư mục tax/ thay đổi
3. Job build-java chạy: Maven build → Docker build → push image
4. Image được push lên Docker Hub: <DOCKER_USER>/tax:<commit_sha>
5. Có thể kiểm tra tại: hub.docker.com

Kết quả kiểm thử CI

```
# Kiem tra image tren Docker Hub
docker pull <DOCKER_USER>/tax:abc1234
# Status: Downloaded newer image

# Xac nhan tag correct
```

```
docker image ls | grep tax
# <DOCKER_USER>/tax    abc1234    sha256:...    2 minutes ago
```

3.2 Kiểm thử Developer Build (Yêu cầu 4)

3.2.1 Kịch bản test

1. Vào GitHub → Actions → **developer_build** → **Run workflow**
2. Nhập:
 - service_name: tax
 - branch_name: dev_tax_service
3. Click **Run workflow**

3.2.2 Kết quả

```
# Kiểm tra namespace developer
kubectl get pods -n developer
# NAME                                READY   STATUS    RESTARTS   AGE
# cart-xxx                            2/2     Running   0           2m
# order-xxx                           2/2     Running   0           2m
# tax-xxx                             2/2     Running   0           2m
#   <- custom image
# ...

# Lay NodePort của tax service
kubectl get svc tax -n developer
# NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
# tax       NodePort    10.43.x.x    <none>         80:3xxxx/TCP     2m

# Thêm vào /etc/hosts trên máy developer
<WORKER_IP> developer.yas.local.com api.developer.yas.local.com

# Test truy cập
curl http://api.developer.yas.local.com/tax/actuator/health
# {"status":"UP"}
```

Listing 3.2: Kiểm tra sau khi developer_build chạy

3.3 Kiểm thử Cleanup (Yêu cầu 5)

```
# Sau khi chạy cleanup_developer_build workflow
kubectl get namespace developer
# Error from server (NotFound): namespaces "developer" not found
# -> Namespace đã bị xóa thành công
```

Listing 3.3: Test cleanup workflow

3.4 Kiểm thử Dev/Staging Deploy (Yêu cầu 6)

3.4.1 Test Dev Environment

```
git checkout main
git merge dev_tax_service
git push origin main
# -> Workflow npt-dev-deploy.yml tu dong chay
# -> ArgoCD ApplicationSet sync -> deploy vao namespace dev
```

Listing 3.4: Trigger dev deploy bằng push vào main

```
kubectl get pods -n dev | grep -c Running
# 20+ <- tat ca service dang Running

curl http://storefront.dev.yas.local.com
# HTTP 200
```

Listing 3.5: Kiểm tra namespace dev

3.4.2 Test Staging Environment

```
git tag v1.0.0
git push origin v1.0.0
# -> Workflow npt-staging-deploy.yml chay
# -> Build tat ca services voi tag :v1.0.0
# -> Deploy vao namespace staging

kubectl get pods -n staging
# All running with image tag v1.0.0
```

Listing 3.6: Trigger staging deploy bằng tag

3.5 Kiểm thử Istio mTLS (Nâng cao)

3.5.1 Kiểm tra mTLS đang hoạt động

```
# Kiem tra PeerAuthentication
kubectl get peerauthentication -n yas
# NAME          MODE      AGE
# default-mtls  STRICT    1d

# Kiem tra pod co 2/2 containers (co sidecar)
kubectl get pods -n yas | grep "2/2"
# Tat ca pods phai co 2/2 Running

# Verify bang istioctl
./istio-1.24.3/bin/istioctl experimental authz check \
$(kubectl get pod -n yas -l app.kubernetes.io/name=product \
-o jsonpath='{.items[0].metadata.name}') -n yas
```

Listing 3.7: Verify mTLS STRICT mode

3.5.2 Kiểm tra Authorization Policy

```
# Test 1: order duoc phep goi tax (co trong policy)
kubectl exec -n yas deploy/order -- \
  curl -v http://tax/tax/actuator/health
# -> HTTP 200 OK (THANH CONG - duoc phep)

# Test 2: search khong duoc phep goi tax (khong co trong policy)
kubectl exec -n yas deploy/search -- \
  curl -v http://tax/tax/actuator/health
# -> RBAC: access denied
# -> HTTP 403 Forbidden (THANH CONG - bi chan)
```

Listing 3.8: Test Authorization Policy

3.5.3 Kiểm tra Retry Policy

```
# Xem log Envoy de thay retry
kubectl logs deploy/order -n yas -c istio-proxy | grep "retry\|
  upstream"
# upstream_request_count: 3 <- thu lai 3 lan

# Kiem tra config retry tren VirtualService
kubectl get virtualservice tax-retry -n yas -o yaml
```

Listing 3.9: Test Retry Policy

3.5.4 Kiali Topology

```
# Port-forward Kiali
kubectl port-forward svc/kiali -n istio-system 20001:20001 &

# Truy cap: http://localhost:20001
# Menu: Graph -> Namespace: yas
# Bat: Display -> Security (hien thi lock icon cho mTLS)
# Ket qua: Tat ca service-to-service connection hien lock icon xanh =
  mTLS active
```

Listing 3.10: Truy cập Kiali và kiểm tra topology

Kết quả Kiali sau khi hoàn thiện cấu hình

- Tất cả 21 DestinationRule hiển thị [OK] **xanh** (không còn wildcard host)
- Tất cả 18 Service hiển thị [OK] **xanh** (đã fix port name `http-metric`)
- AuthorizationPolicy hiển thị đúng workload (đã sửa selector sang

app.kubernetes.io/name)

- Service mesh topology hiển thị đầy đủ traffic flow với mTLS lock icon

3.6 Kiểm thử End-to-End

```
# Storefront (ung dung chinh)
curl -o /dev/null -w "%{http_code}" http://storefront.yas.local.com
# -> 200

# Backoffice
curl -o /dev/null -w "%{http_code}" http://backoffice.yas.local.com
# -> 302 (redirect den Keycloak login)

# Keycloak (Identity)
curl -o /dev/null -w "%{http_code}" http://identity.yas.local.com
# -> 302

# pgAdmin
curl -o /dev/null -w "%{http_code}" http://pgadmin.yas.local.com
# -> 302

# AKHQ (Kafka UI)
curl -o /dev/null -w "%{http_code}" http://akhq.yas.local.com
# -> 307

# Grafana (Observability)
curl -o /dev/null -w "%{http_code}" http://grafana.yas.local.com
# -> 302
```

Listing 3.11: Test tổng thể các URL hệ thống

```
kubectl get pods -A --no-headers | awk '{print $4}' | sort | uniq -c |
sort -rn
# 56+ Running
# 3 Completed <- ingress admission jobs, binh thuong
# 0 Error / CrashLoopBackOff / Pending
```

Listing 3.12: Kiểm tra tổng thể pods

Chapter 4

Vấn đề gặp phải và Cách khắc phục

4.1 Vấn đề Infrastructure

4.1.1 PostgreSQL cluster không tạo được — Helm template syntax error

Triệu chứng: Sau khi chạy `setup-cluster.sh`, chỉ có `postgres-operator` pod Running nhưng không có pod database nào.

Nguyên nhân: Trong file `k8s/deploy/postgres/postgresql/templates/postgresql.yaml`, hai dòng cấu hình có khoảng trắng thừa trong cặp ngoặc nhọn Helm:

```
# SAI: Helm coi đây là string literal, không phải template expression
recommendation: { { .Values.username } }
webhook:        { { .Values.username } }

# ĐÚNG:
recommendation: {{ .Values.username }}
webhook:        {{ .Values.username }}
```

Listing 4.1: Lỗi Helm template syntax

Cách khắc phục:

```
sed -i 's/{ { .Values.username } }/{{ .Values.username }}/g' \
  k8s/deploy/postgres/postgresql/templates/postgresql.yaml
helm upgrade --install postgres ./postgres/postgresql \
  --namespace postgres --set replicas=1 --set username=yasadminuser
```

4.1.2 Strimzi Kafka — Chuỗi 3 vấn đề tương thích

Vấn đề 2a: Strimzi 0.51.0 không hỗ trợ ZooKeeper mode

Repo gốc dùng Kafka với ZooKeeper mode. Strimzi từ v0.46.0 đã loại bỏ hoàn toàn hỗ trợ ZooKeeper, chỉ còn KRaft mode.

Vấn đề 2b: Strimzi 0.44.0/0.45.0 không tương thích K8s 1.35


```
# Logs Strimzi 0.44.0 tren K8s 1.35:
java.lang.NoSuchFieldError: emulationMajor
  at io.fabric8.kubernetes.client...
# -> fabric8 library cu chua ho tro field moi cua K8s 1.35 API
```

Vấn đề 2c: Strimzi 0.51.0 + Kafka version sai

```
# Logs Strimzi 0.51.0:
"Unsupported Kafka version 3.9.0. Supported versions are: [4.0.0,
  4.1.0]"
```

Giải pháp cuối cùng: Chuyển sang Strimzi 0.51.0 + Kafka 4.1.0 + KRaft mode.

```
# KafkaNodePool (moi - thay the ZooKeeper + replicas + storage)
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaNodePool
metadata:
  name: kafka-cluster-dual-role
  labels:
    strimzi.io/cluster: kafka-cluster
spec:
  replicas: 1
  roles:
    - controller    # Thay the ZooKeeper
    - broker
  storage:
    type: persistent-claim
    size: 10Gi
---
# Kafka (sua - them annotations KRaft, bo zookeeper section)
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata:
  name: kafka-cluster
  annotations:
    strimzi.io/kraft: enabled          # BAT BUOC
    strimzi.io/node-pools: enabled    # BAT BUOC
spec:
  kafka:
    version: "4.1.0"                  # DOI TU 3.9.0
    # KHONG CON spec.zookeeper
```

Listing 4.2: Cấu trúc kafka-cluster.yaml sau khi sửa

4.1.3 Promtail CrashLoopBackOff — “too many open files”

Nguyên nhân: Promtail dùng inotify để theo dõi file log. Giới hạn mặc định quá thấp dẫn đến crash.

Cách khắc phục:

```
# Tren Minikube (phai lam lai sau moi restart):
minikube ssh -- "sudo sysctl -w fs.inotify.max_user_instances=1024"
```

```
# Tren K3s (chỉ cần làm 1 lần, persist vĩnh viễn):
echo "fs.inotify.max_user_watches=524288" | sudo tee -a /etc/sysctl.d
/99-k3s.conf
sudo sysctl --system
kubectl rollout restart daemonset/promtail -n observability
```

4.1.4 ArgoCD ApplicationSet Controller CrashLoopBackOff

Nguyên nhân: CRD ApplicationSet chưa được đăng ký trong cluster. Khi dùng `kubectl apply` thông thường để apply CRD ArgoCD (>256KB), gặp lỗi annotation size limit.

Cách khắc phục:

```
# Dùng --server-side để bypass annotation size limit
kubectl apply -k "https://github.com/argoproj/argo-cd/manifests/crds?
ref=stable" \
--server-side

kubectl rollout restart deployment/argocd-applicationset-controller -n
argocd
```

Giải thích: Flag `--server-side` chuyển logic apply sang API server, không lưu annotation `last-applied-configuration` → không bị giới hạn 262144 bytes.

4.1.5 Loki chart mới yêu cầu `schema__config`

Nguyên nhân: Loki chart 6.x bắt buộc có `schema_config`. File `loki.values.yaml` gốc thiếu trường này.

Cách khắc phục:

```
helm upgrade --install loki grafana/loki \
--namespace observability \
-f ./observability/loki.values.yaml \
--set loki.useTestSchema=true # Tự tạo schema mặc định
```

4.1.6 Prometheus/Grafana — leaked secrets validation

Nguyên nhân: Grafana chart mới có security validation, từ chối field `database.password` được định nghĩa trực tiếp trong values.

Cách khắc phục:

```
helm upgrade --install prometheus prometheus-community/kube-prometheus
-stack \
--namespace observability \
-f ./observability/prometheus.values.yaml \
--set grafana.assertNoLeakedSecrets=false
```

4.1.7 OpenTelemetry Collector — receiver/exporter “loki” không tồn tại

Nguyên nhân: OTel Collector image mới không bundle loki receiver/exporter. Thêm vào đó, logging exporter bị rename thành debug và API v1alpha1 đã deprecated.

Table 4.1: Thay đổi cấu hình OpenTelemetry Collector

Cũ	Mới	Lý do
receivers.loki	Xóa	Không có trong standard image
exporters.loki	exporters.otlphttp/1.0	Dùng OTLP gửi đến Loki gateway
exporters.logging	exporters.debug	logging bị rename
processors.batch:	processors.batch:	Tránh null object warning
null	{}	
v1alpha1	v1beta1	v1alpha1 deprecated

Cách khắc phục: Phải xóa CR cũ trước khi reinstall (tránh Helm strategic merge giữ config cũ):

```
kubectl delete opentelemetrycollectors opentelemetry -n observability
helm uninstall opentelemetry-collector -n observability
helm upgrade --install opentelemetry-collector ./observability/
  opentelemetry \
  --namespace observability
```

4.2 Vấn đề K3s và Chuyển đổi từ Minikube

4.2.1 Cluster quá tải — API server ngừng hoạt động (Minikube)

Nguyên nhân: Minikube chạy trong Docker container bị giới hạn RAM (~4GB). Sau khi cài Loki (9 pods mới) + Prometheus (6 pods), hệ thống vượt quá giới hạn → kernel OOM killer tắt kubelet và API server.

Cách khắc phục: Chuyển sang K3s (overhead chỉ ~512MB), thêm worker node để phân phối workload.

4.2.2 K3s agent không start — port 6444 đang bị chiếm

Triệu chứng:

```
systemctl status k3s-agent
# -> listen tcp 0.0.0.0:6444: bind: address already in use
```

Cách khắc phục:

```
# Tìm process đang dùng port 6444
ss -tlnp | grep 6444
# Kill process cu
kill -9 <PID>
sudo systemctl restart k3s-agent
```

4.2.3 Firewall Fedora block inter-pod traffic

Triệu chứng: ingress-nginx trả 502, logs có connect() failed (113: Host is unreachable).

Nguyên nhân: Fedora firewalld không tự nhận interface CNI của K3s.

Cách khắc phục:

```
sudo firewall-cmd --permanent --zone=trusted --add-interface=cni0
sudo firewall-cmd --permanent --zone=trusted --add-interface=flannel.1
sudo firewall-cmd --permanent --zone=trusted --add-source=10.42.0.0/16
sudo firewall-cmd --permanent --zone=trusted --add-source=10.43.0.0/16
sudo firewall-cmd --reload
```

4.3 Vấn đề Istio Service Mesh

4.3.1 502 Bad Gateway sau khi bật mTLS STRICT

Nguyên nhân: ingress-nginx proxy qua Pod IP (PassthroughCluster) thay vì service identity → DestinationRule không được áp dụng → mTLS fail.

Các hướng đã thử:

Không Đổi toàn namespace sang PERMISSIVE — không đạt yêu cầu

Không Dùng portLevelMtls — không áp dụng được namespace-wide

OK Inject sidecar cho ingress-nginx + export DestinationRule + bật service-upstream

4.3.2 Kiali cảnh báo KIA0601 — Port name sai format

Nguyên nhân: Port metrics của backend service có name: metric, Istio yêu cầu tên port phải bắt đầu bằng protocol.

Cách khắc phục:

```
# Patch tat ca 18 service đang live
for svc in cart customer inventory location media order product \
    promotion rating recommendation sampledata search \
    storefront-bff backoffice-bff tax webhook payment payment-
    paypal; do
    kubectl patch svc "$svc" -n yas --type='json' \
        -p='[{"op":"replace","path":"/spec/ports/1/name","value":"http-
        metric"}]'
done
```

4.3.3 Kiali lỗi DestinationRule wildcard host

Nguyên nhân: Kiali v2.x không validate được wildcard host *.yas.svc.cluster.local.

Cách khắc phục: Xóa 1 DestinationRule wildcard, thay bằng 21 DestinationRule riêng cho từng service. Sau khi tách, tất cả hiển thị [OK] xanh trong Kiali.

4.3.4 AuthorizationPolicy không match workload

Nguyên nhân: Policy dùng selector `app: cart` nhưng pod thực tế dùng label `app.kubernetes.io/name: cart` (do Helm chart tạo ra).

Cách khắc phục: Sửa tất cả selector:

```
# Trước:
selector:
  matchLabels:
    app: cart
# Sau:
selector:
  matchLabels:
    app.kubernetes.io/name: cart
```

4.4 Tổng kết

Table 4.2: Tổng kết vấn đề và trạng thái giải quyết

Vấn đề	Trạng thái	Giải pháp
PostgreSQL Helm template syntax	Đã fix	Xóa khoảng trắng trong <code>{{ }}</code>
Strimzi Kafka ZooKeeper/KRaft	Đã fix	KRaft mode + Kafka 4.1.0
Promtail inotify limit	Đã fix	<code>/etc/sysctl.d/99-k3s.conf</code>
ArgoCD CRD size limit	Đã fix	<code>kubectl apply --server-side</code>
Loki schema_config	Đã fix	<code>--set loki.useTestSchema=true</code>
Grafana leaked secrets	Đã fix	<code>assertNoLeakedSecrets=false</code>
OTel Collector loki receiver	Đã fix	Dùng <code>otlphttp/loki</code> exporter
Cluster OOM (Minikube)	Giải quyết	Chuyển sang K3s + worker nodes
K3s agent port conflict	Đã fix	Kill process cũ + restart agent
Fedora firewall block CNI	Đã fix	<code>firewall-cmd trusted zone</code>
502 Bad Gateway (mTLS)	Đã fix	<code>service-upstream</code> + sidecar injection
Kiali KIA0601 port name	Đã fix	Rename <code>metric</code> → <code>http-metric</code>
Kiali wildcard DestinationRule	Đã fix	Tách thành 21 DR riêng
AuthorizationPolicy selector	Đã fix	Dùng <code>app.kubernetes.io/name</code>
Debezium Connect crash	Bỏ qua	Không ảnh hưởng đến đồ án CD